

UNIT 3 – IMP QUESTIONS + PYQ QUESTIONS

1. What are the different types of file organizations used in a DBMS?
2. Compare and contrast various file organizations based on their efficiency for different database operations.
3. What is the role of indexing in databases, and how does it improve performance?
4. Explain Hash-based indexing in detail.
5. Explain Tree-based indexing in detail.
6. What is a B+ Tree? Explain its structure and characteristics with a neat diagram.
7. Why is a B+ Tree classified as a "Dynamic Index Structure"?
8. Give the algorithm to insert an element into a B+ Tree.
9. What is a Query Optimizer? Explain its working with a neat diagram.
10. What is a database transaction?
11. Briefly explain the ACID properties of a transaction.
12. What is a schedule in the context of transaction management?
13. Illustrate the anomalies associated with Interleaved Execution and explain them in detail.
14. What is the fundamental concept behind Lock-Based Concurrency Control?
15. What is the Two-Phase Locking (2PL) protocol?
16. Explain the Strict Two-Phase Locking (Strict 2PL) protocol in detail.
17. What are the 2 rules of Strict 2PL? Briefly explain them.

Part 1: Storage and Indexing Basics

1. What are the different types of file organizations used in a DBMS?

File organization refers to the way data records are physically arranged and stored in a file on the disk. The primary types include:

- **Heap (Unordered) Files:** Records are placed sequentially at the end of the file as they are inserted. There is no specific ordering of records.
 - *Why use it:* It is excellent for fast, bulk data insertion (like a daily log file).
 - *Drawback:* It is highly inefficient for searching, as finding a record requires a full table scan.
- **Sorted Files:** Records are ordered sequentially based on the value of a specific field (the search key), much like a telephone directory.
 - *Why use it:* This allows for highly efficient range queries and fast binary searches.
 - *Drawback:* Maintaining the sorted order makes insertions and deletions expensive because records must be shifted to make room.
- **Hashed Files:** A mathematical hash function is applied to a specific field (the hash key) to determine the exact disk block (bucket) where the record should be stored.
 - *Why use it:* This is exceptionally fast for equality searches (e.g., finding a student by their exact ID).
 - *Drawback:* It does not support range queries efficiently because hashing scatters data randomly.

2. Compare and contrast various file organizations based on their efficiency for different database operations.

To secure 8 marks, present this comparison as a structured evaluation of costs:

- **Scan (Fetch all records):** * *Heap:* Fast, as records are stored contiguously.
 - *Sorted:* Fast, as it just reads start to finish.
 - *Hashed:* Can be slower if buckets are not completely full, requiring the system to read empty space.
- **Equality Search (Find specific record like ID = 5):**
 - *Heap:* Very slow. Requires a full linear scan, examining half the pages on average.
 - *Sorted:* Moderate to Fast. Allows Binary Search, which skips huge chunks of data.
 - *Hashed:* Very Fast. The hash function acts as a direct shortcut to the exact page.
- **Range Search (Find records between values like IDs 10 to 50):**
 - *Heap:* Very slow. Requires a full linear scan of the entire disk.

- *Sorted*: Very Fast. Find the first record, then just scan sequentially forward.
- *Hashed*: Very slow. Hashing destroys ordering, so it forces a full scan of all buckets.
- **Insertion (Adding a new record):**
 - *Heap*: Very Fast. Just append to the very end.
 - *Sorted*: Slow. Requires finding the correct spot and potentially shifting many records down.
 - *Hashed*: Fast. Apply the hash function and insert into the calculated bucket.

3. What is the role of indexing in databases, and how does it improve performance?

An index is an auxiliary data structure designed to speed up the retrieval of records in response to certain search conditions.

- **Role:** An index acts exactly like a table of contents at the back of a textbook. Instead of reading the entire database file page-by-page to find a specific row (a full table scan), the DBMS searches the much smaller index, which provides a direct pointer (Record ID) to the actual data location on the disk.
- **How it improves performance:**
 - *Reduces I/O Operations*: Disk reading is the slowest operation in a computer. Indexes drastically reduce the number of disk pages that need to be read into memory.
 - *Optimizes Sorting*: If an index exists on a column used in an ORDER BY clause, the database can fetch the data in the already sorted order, bypassing a costly sorting step entirely.
 - *Enforces Integrity*: Unique indexes automatically enforce PRIMARY KEY constraints efficiently without checking every single row.

4. Explain Hash-based indexing in detail.

Hash-based indexing groups records into "buckets" based on a hashing algorithm.

- **Mechanism:** A hash function is applied to the search key. The mathematical result of this function specifies the bucket (a disk block) where the data entry is safely stored.
- **Characteristics:**
 - It is best suited for equality selections (e.g., SELECT * WHERE ID = 105).
 - It cannot be used for range selections (e.g., WHERE ID > 100) because the hash function scatters the keys randomly, destroying any logical sorting.
- **Types:**
 - *Static Hashing*: The number of buckets is fixed from the start. If a bucket gets full, "overflow pages" are linked to it. Long overflow chains slow down searches.
 - *Dynamic Hashing*: The number of buckets grows and shrinks automatically as the database changes size. This keeps search times constantly fast and eliminates the need for long overflow chains.

5. Explain Tree-based indexing in detail.

Tree-based indexing organizes data entries in a hierarchical, sorted tree structure.

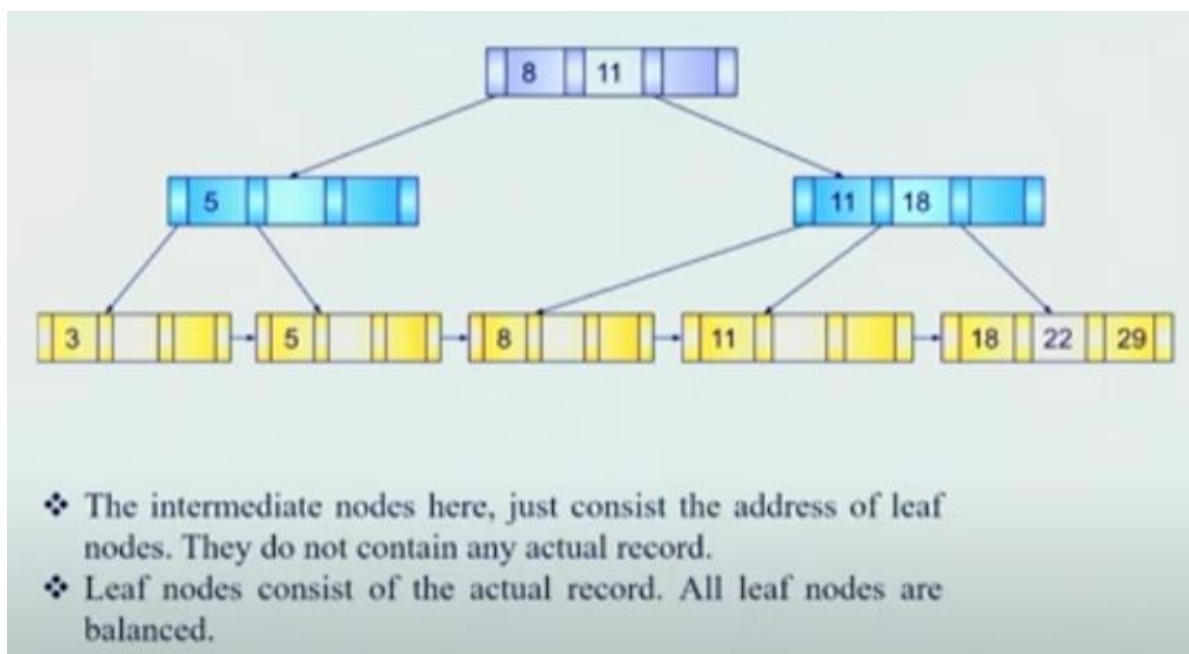
- **Mechanism:** The data structure directs a search for a record by navigating from the top (root) of the tree down to the bottom (leaves). Each internal node contains search keys that act as simple routing guides (like "go left for numbers under 50, go right for numbers over 50").
- **Characteristics:**
 - Highly efficient for range selections (e.g., finding employees with salaries between 50,000 and 70,000) as well as equality searches.
 - The leaf nodes at the bottom are sequentially linked together. This allows the DBMS to find the start of a range and simply walk forward to get the rest.
- **Examples:** ISAM (Indexed Sequential Access Method) which is static, and the B+ Tree, which is the most widely used dynamic tree index in modern databases.

Part 2: Deep Dive into B+ Trees

6. What is a B+ Tree? Explain its structure and characteristics with a neat diagram.

A B+ tree is a balanced search tree widely used as a dynamic index structure in databases. It keeps data sorted and allows searches in a very small number of steps.

- **Structure & Characteristics:**
 - *Balanced:* All paths from the root down to a leaf node are of the exact same length. This guarantees predictable and consistent search times for every query.
 - *Internal Nodes:* Act solely as a directory/router to guide searches. They contain keys and pointers to child nodes but hold absolutely no actual data records.
 - *Leaf Nodes:* Located at the very bottom, these contain the actual data entries (or pointers to the records).
 - *Sequential Linking:* All leaf nodes are connected horizontally via a doubly linked list. This makes sequential scanning and range queries extremely fast.



7. Why is a B+ Tree classified as a "Dynamic Index Structure"?

A B+ Tree is "dynamic" because it automatically adjusts its shape to remain perfectly balanced as the database naturally grows or shrinks over time.

- **Self-Balancing:** Unlike older static indexes that degrade over time and require a human administrator to manually rebuild them, a B+ tree dynamically manages its own structure during everyday operations.
- **Node Splitting and Merging:** If a node overflows because you inserted too much data, it automatically splits into two nodes, and a key is pushed up to the parent. If a node becomes too empty due to deletions, it merges with a neighboring sibling node to save space.
- **Performance Guarantee:** Because it adapts dynamically, the tree always stays relatively short and flat. This ensures that searching even a massive database takes very few steps.

8. Give the algorithm to insert an element into a B+ Tree.

For 8 marks, clearly list the steps of the recursive insertion process.

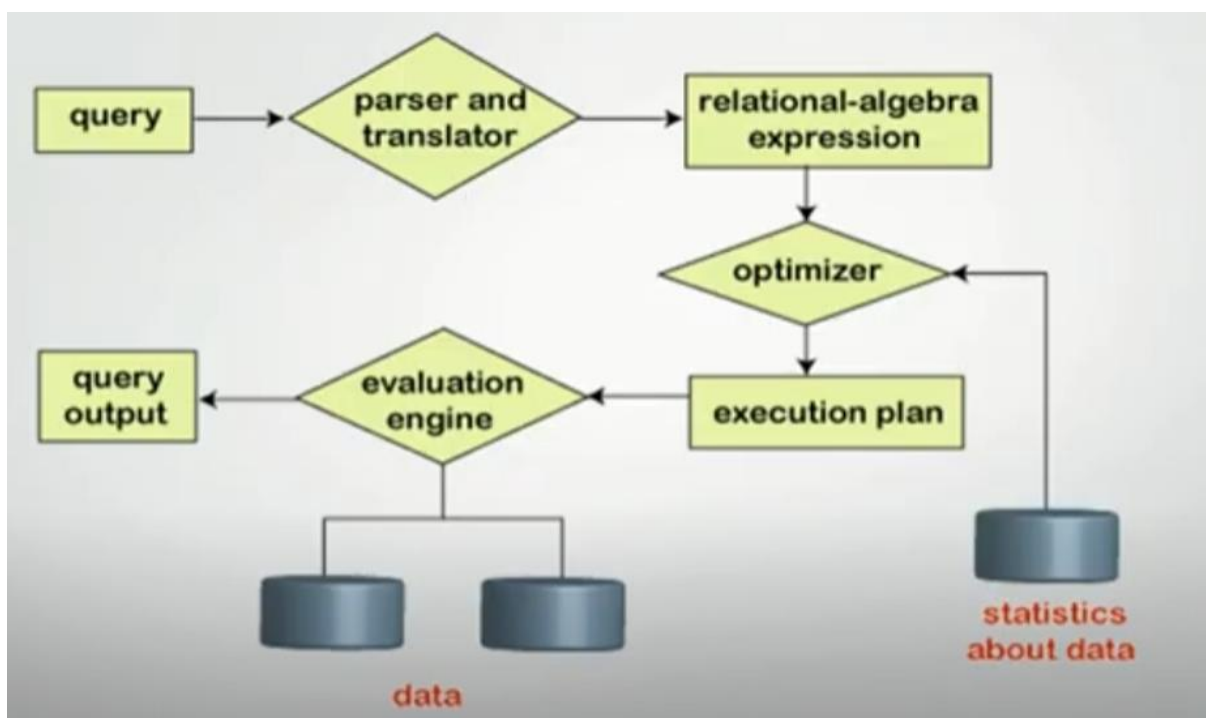
1. **Find the Target Leaf:** Traverse the tree from the root downwards to find the correct leaf node (**L**) where the new data entry naturally belongs based on its sorted value.
2. **Insert into Leaf:** * If **L** has enough empty space, insert the entry in sorted order. **Done.**
 - If **L** is full and has no space, proceed to **Split**.
3. **Split the Leaf Node:**
 - Create a brand new leaf node (**L2**).
 - Redistribute the entries evenly between **L** and **L2**.
 - *Crucial step for leaves:* Copy up the middle key (the smallest key in **L2**) to the parent node so the parent knows how to route traffic. Link **L** and **L2** horizontally.
4. **Insert into Parent:**
 - Insert the copied-up key and a pointer to **L2** into the parent node.
 - If the parent node has space for this new key, **Done**.
 - If the parent node is also full, split the parent node.
5. **Split Internal Node (if necessary):**
 - Create a new internal node and redistribute the keys.
 - *Crucial step for internal nodes:* Push up the middle key to its parent (do not keep a copy in the internal node).
 - Repeat this process recursively up the tree. If the root splits, create a new root on top, increasing the height of the tree by one.

Part 3: Query Processing

9. What is a Query Optimizer? Explain its working with a neat diagram.

A Query Optimizer is a critical, built-in component of a database management system that determines the fastest and most efficient way to execute a given SQL query.

- **Working:** SQL only tells the database *what* data you want, not *how* to get it. The optimizer's job is to figure out the "how." It translates the SQL query into an internal format. It then generates multiple alternative "execution plans" (e.g., Should I use an index? Should I scan the whole table? Which table should I join first?). It looks at statistical metadata (like table sizes) to estimate the CPU and I/O cost of each plan. Finally, like a GPS finding the fastest route, it selects the plan with the lowest estimated cost to run.
- **Diagram Blueprint (Draw this in your exam):**



Part 4: Transaction Management Basics

10. What is a database transaction?

A transaction is the DBMS's abstract view of a user program. It is a logical unit of work consisting of one or more database operations (like reading, inserting, updating, or deleting records) bundled together into a single package.

- **The Core Rule:** To the system and the user, a transaction executes completely, or it does not execute at all. It acts as a safety net against system crashes.
- **Transaction States:** To get full marks, mention that a transaction passes through these states:
 1. *Active:* The transaction is currently running its instructions.
 2. *Committed:* The transaction finished successfully, and changes are permanently saved.

3. *Failed/Aborted*: An error occurred, and the system rolls back to the original state.
- **Real-World Example**: Transferring money from Account A to Account B. This requires two distinct operations: deducting money from A, and adding money to B. A transaction ensures both must succeed together. If the power goes out after deducting from A but before adding to B, the transaction rolls back so the money isn't lost.

11. Briefly explain the ACID properties of a transaction.

For 8 marks, explicitly define each property and state which DBMS part handles it.

- **Atomicity ("All or Nothing")**: A transaction is an indivisible unit. Either all of its actions are executed completely, or none of them are. If a transaction fails mid-execution, the database automatically undoes any partial changes. *(Handled by the DBMS Recovery Manager)*.
- **Consistency**: A transaction must transition the database from one valid state to another valid state. It guarantees that all database constraints and business rules (like "an account balance cannot drop below zero") are strictly obeyed. *(Handled by the Application Programmer)*.
- **Isolation**: Concurrent transactions execute without stepping on each other's toes. The system ensures that the execution of concurrent transactions yields the exact same safe result as if they were executed serially (one by one in a vacuum). *(Handled by the Concurrency Control Manager)*.
- **Durability**: Once a transaction successfully finishes and says "committed," its changes are permanent. They will survive any subsequent system crashes, power outages, or hardware failures. *(Handled by the Recovery Manager using system logs)*.

12. What is a schedule in the context of transaction management?

A schedule (or history) is simply a sequential timeline of actions (reads, writes, commits, and aborts) from a set of multiple transactions running at the same time.

- **Purpose**: It defines the exact chronological order in which the database executes operations to ensure data doesn't get corrupted.
- **Serial Schedule**: A schedule where transactions are executed completely one after another.
 - *How it works*: T1 finishes completely, and only then does T2 begin.
 - *Pros/Cons*: It is 100% safe from data errors, but very slow because it forces users to wait in a single-file line.
- **Interleaved (Concurrent) Schedule**: The operations of different transactions overlap in time.
 - *How it works*: Much like a chef working on two different meals simultaneously to save time, the CPU switches back and forth between T1 and T2.
 - *Pros/Cons*: This heavily improves system speed and resource utilization, but requires strict rules (like locking) to prevent data corruption.

13. Illustrate the anomalies associated with Interleaved Execution and explain them in detail.

When transactions overlap without strict rules, three major data errors (anomalies) can happen:

- **Write-Read Anomaly (Dirty Read):** Occurs when Transaction T2 reads a value that was modified by T1, but T1 hasn't officially finished (committed) yet.
 - *Timeline Example:* T1 deducts 100 from Account A. T2 reads this new balance to calculate interest. Suddenly, T1 crashes and aborts. T2 is now running calculations on a fake, "dirty" balance that technically never existed.
- **Read-Write Anomaly (Unrepeatable Read):** Occurs when Transaction T1 reads a value twice, but gets a different result the second time because T2 modified it in between.
 - *Timeline Example:* T1 checks available flight seats and sees 5. T2 quickly books a seat and commits (seats now 4). T1 attempts to finally book a ticket based on the '5' it saw earlier, causing a system inconsistency.
- **Write-Write Anomaly (Lost Update):** Occurs when two transactions modify the exact same value at the same time, and one update blind-sides the other.
 - *Timeline Example:* T1 modifies an inventory count. Before T1 commits, T2 modifies the exact same inventory count. T1's update is completely wiped out and overwritten by T2.

Part 5: Concurrency Control (Advanced Transaction Management)

14. What is the fundamental concept behind Lock-Based Concurrency Control?

Lock-based concurrency control is a strict traffic-cop mechanism used to prevent the anomalies of interleaved execution (like Dirty Reads) by restricting access to data.

- **The Core Rule:** Before a transaction can read or write to a database object (like a row or a table), it must officially request a "lock" on that object from the system's Lock Manager.
- **Shared (S) Locks:** Requested when a transaction wants to *read* an object.
 - *Compatibility:* Multiple transactions can hold a Shared lock on the same object simultaneously (just like many people can read a public notice board at once without interfering with each other).
- **Exclusive (X) Locks:** Requested when a transaction wants to *write/modify* an object.
 - *Compatibility:* Only one transaction can hold an Exclusive lock at a time. If an X lock is held, nobody else can read or write to that data until the lock is dropped.

15. What is the Two-Phase Locking (2PL) protocol?

Two-Phase Locking is a set of rules that guarantees transactions can safely overlap without causing data corruption. It dictates that a transaction must acquire and release its locks in two distinct, strict phases:

- **Phase 1: Growing Phase:** The transaction may request and obtain locks (Shared or Exclusive) as it needs them to do its work.

- *Strict Rule:* It is strictly forbidden from releasing any locks during this phase. (Think of it as gathering all the tools needed for a job).
- **The Lock Point:** The exact moment a transaction has acquired all the locks it will ever need.
- **Phase 2: Shrinking Phase:** Once the transaction releases its very first lock, it officially enters the shrinking phase.
 - *Strict Rule:* It may continue to release locks, but it is strictly forbidden from acquiring any *new* locks. (Think of it as putting tools back; the job is winding down).

16. Explain the Strict Two-Phase Locking (Strict 2PL) protocol in detail.

Strict 2PL is an upgraded, safer version of the standard 2PL protocol. It is the industry standard because it is designed specifically to solve a major system problem called "Cascading Aborts."

- **The Problem with Basic 2PL:** In standard 2PL, a transaction could release a write-lock during its shrinking phase *before* it actually committed. This was dangerous because it allowed another transaction to peek at uncommitted data (causing a dirty read). If the first transaction failed, every transaction that peeked at its data also had to be aborted (a cascading abort).
- **How Strict 2PL solves this:** It forces a transaction to hold onto all of its Exclusive (write) locks until the very end of the transaction. The database guarantees that no other transaction can see or touch an object modified by transaction T until T has officially saved (committed) or rolled back (aborted).

17. What are the 2 rules of Strict 2PL? Briefly explain them.

To successfully implement Strict 2PL and guarantee a safe database, a transaction must blindly follow two absolute rules:

- **Rule 1 (The Request Rule):** If a transaction wants to read an object, it must first obtain a Shared (S) lock. If it wants to modify an object, it must first obtain an Exclusive (X) lock. It cannot bypass the lock manager to access data.
- **Rule 2 (The Hold Rule):** All Exclusive (X) locks held by a transaction are held hostage and released **only when the transaction completely finishes** (i.e., at the exact moment it commits or aborts). It cannot release them early just because it is done editing that specific row.
 - *Benefit:* This simple rule ensures that "Dirty Reads" are physically impossible, making the database highly reliable.